

RuleUp 관측 이벤트와 퍼널

Amplitude · Firebase 에서 무엇을 볼 수 있는가 · 2026-09-11 · develop 기준

이벤트 수	37종 — 온보딩 10 · 챌린지 22 · 자동 5
진실의 출처	코드다. 각 feature domain 의 observability/<Feature>Events.kt . 팩토리 시그니처가 곧 스키마이고 골든 테스트가 고정한다.
샘플링	행동 이벤트에는 없다. 발생한 만큼 다 나간다. 심각도 하한은 진단 채널에만 있다.
두 출구	Amplitude 와 Firebase 가 병행한다. 같은 이벤트가 두 곳에 쌓이므로 집계할 때 출처를 섞지 않는다.

1 어디로 흘러가는가

ViewModel 이 Observability.log(channel) { payload } 를 부르면 파이프라인이 채널별로 출구를 고른다.

채널	무엇	Firebase Analytics	Amplitude	Crashlytics
BUSINESS	사용자 행동	○	○	-
PERFORMANCE	TTI · 잼 · 메모리	○	○	-
DIAGNOSTIC	에러 · 경고	-	-	○ (WARN 이상)

모든 이벤트에 screen 이 붙는다
이벤트가 난 시점의 화면 경로다. 성능·진단 페이로드에는 화면 필드가 따로 없어서 이 값이 유일한 출처다. Amplitude 키가 비면 Amplitude 출구를 아예 달지 않는다. Firebase 로는 계속 나간다.

2 온보딩 퍼널

가입 과정을 단계로 쪼개 어디서 빠져나가는지 보는 갈때기. 출처 OnboardingEvents.kt

login_screen_view entry_type = fresh	완주율의 분모. fresh 로 걸러야 재로그인이 섞이지 않는다
login_attempt	소요 시간 측정의 시작점
login_result success = true	실패는 error_code 로 사유가 갈린다
onboarding_step_complete nickname	여기부터 6단계. step 필터로 6번 넣는다
... interest · birth · gender · photo	skipped 속성이 관심사·사진 선택률을 만든다
onboarding_step_complete terms	약관이 마지막 단계
signup_complete	완주율의 분자. duration_ms 중앙값이 가입 소요 시간

이벤트	언제	속성
login_screen_view	로그인 화면 진입	entry_type (fresh / relogin)
login_attempt	소셜 버튼 클릭	provider
login_result	토큰 발급 성공·실패	provider, success, error_code, is_new_user, restored
onboarding_step_view	6단계 각 진입	step, step_index (1부터)

이벤트	언제	속성
onboarding_step_complete	6단계 각 완료	step, skipped
nickname_check	닉네임 확인 응답	valid, available, reason
signup_complete	가입 성공	interest_count, has_gender, optional_agreements, duration_ms
signup_failed	가입 실패	error_code
profile_image_upload_result	사진 등록 결과	success, error_code
session_expired	세션이 끊겨 로그인 복귀	trigger (other_device / expired)

step 값은 순서대로 nickname · interest · birth · gender · photo · terms. **step_index** 는 1부터 센다 — 화면의 n/6 표기와 같은 기준이라야 집계와 화면이 어긋나지 않는다. 회색으로 적은 속성은 **해당 상황에서만 실린다**. 성공이면 error_code 키를 아예 넣지 않는다 — 빈 문자열은 집계에 가짜 분류를 하나 만든다.

3 탐색 → 참여 퍼널

출처 ChallengeEvents.kt

challenge_id 가 축이다
노출 → 클릭 → 상세 → 참여까지 같은 값으로 이어진다. 그게 전환율 계산을 성립시킨다.

explore_home_view	전환율의 분모
challenge_card_impression	뷰포트 50% · 1초 이상만. 실제로 본 카드 수다
challenge_card_click	source 로 인기 섹션과 목록을 나눈다
challenge_detail_view	상세→참여 전환의 분모
challenge_join_attempt	게이트에 막히기 전 시점
challenge_join_result	전환율의 분자. 실패는 error_code 가 게이트 별 이탈

이벤트	언제	속성
explore_home_view	탐색 홈 진입	has_trending
trending_impression	인기 섹션 노출 (섹션당 1회)	challenge_ids, rank_range
category_grid_click	카테고리 타일 클릭	category, challenge_count
explore_list_view	목록 진입	entry (all/category), sort, filters
explore_filter_apply	필터 적용	categories, verify_type, eligible_only, result_count
explore_sort_change	정렬 변경	sort_from, sort_to, result_count
explore_empty_result	결과 0건	filters, sort
explore_list_load_more	다음 페이지	page_index, sort
challenge_card_impression	카드 노출	challenge_id, position, sort, is_full, eligible, has_metrics
challenge_card_click	카드 클릭	challenge_id, position, source (trending/list), sort
challenge_detail_view	공개 상세 진입	challenge_id, eligible, is_full, source
challenge_join_attempt	참여 버튼 클릭	challenge_id, eligible, is_full

이벤트	언제	속성
challenge_join_result	참여 성공·실패	challenge_id, success, error_code
challenge_clone_click	템플릿 복제	challenge_id

challenge_join_result 의 error_code 에는 가입 차단 사유가 들어간다. TIER_GATE · FULL · REJOIN_COOLDOWN 같은 값의 분포가 곧 게이트별 이탈이다.

4 방 내부 · 생성

방 내부

이벤트	언제	속성
room_view	방 내부 진입	challenge_id, my_role, owner_type
thread_scroll	피드 다음 페이지	page_index, item_count
ranking_view	랭킹 조회	scope (IN_ROOM/CROSS), my_rank_null
room_empty_state_view	피드 빈 상태	owner_type
owner_claim	봇방장 클레임 결과	challenge_id, success, error_code

room_view 는 방 주간 방문율의 분자다. 공지·댓글이 빠지면서 방 안에 흔적을 남길 수단이 사라져, "인증 피드와 랭킹만으로 재방문이 유지되는가"를 판단 하는 유일한 근거가 됐다. ranking_view.my_rank_null 이 높으면 등재 기준 10회가 너무 높다는 신호다.

생성

이벤트	언제	속성
create_start	생성 화면 진입	entry (home / challenge_list_empty / explore_empty / unknown)
create_path_select	경로 선택	path (TEMPLATE / PROMPT)
draft_edit	확인 화면 항목 수정	field, auto_to_manual

생성 퍼널은 반쪽이다
"생성 성공" 이벤트가 클라에 없고 서버 담당이라, 앱 이벤트만으로는 create_start → create_path_select 까지밖에 못 잇는다.
draft_request · draft_result · challenge_create_result · moderation_result 는 BE 로 배정됐다.

5 자동으로 나가는 이벤트

feature 가 부르지 않아도 파이프라인이 보낸다.

이벤트	언제	속성
screen_view	화면 진입마다	screen_name, from_screen
perf_tti	첫 콘텐츠를 그릴 때까지	screen_name, total_millis, outcome
perf_jank	화면당 프레임 창이 닫힐 때	total_frames, janky_frames, frozen_frames, p95_frame_millis
perf_resource	메모리 표본	trigger, heap_used_bytes, heap_max_bytes, low_memory
diagnostic	에러 · 경고	severity, tag, message, error_type, error_hash

`diagnostic` 은 Crashlytics 로만 간다. Amplitude 에서는 보이지 않는다. 파이프라인에 `user_action` 페이로드도 있지만 부르는 곳이 없다 — 대시보드에서 찾지 말 것.

6 무엇을 볼 수 있나

지금 이벤트만으로 만들 수 있는 것들. 괄호는 걸어야 할 속성 필터다.

보고 싶은 것	만드는 법
온보딩 완주율	퍼널: login_screen_view (fresh) → login_attempt → login_result (success) → signup_complete
단계별 이탈	퍼널에 onboarding_step_complete 를 step 필터로 6번 넣는다
가입 소요 시간	signup_complete 의 duration_ms 중앙값
닉네임 병목	nickname_check 를 available 로 나눈다
탐색 → 참여 전환	퍼널: explore_home_view → challenge_card_click → challenge_detail_view → challenge_join_result (success)
인기 섹션 vs 목록	challenge_card_click 을 source 로 나눈다
게이트별 이탈	challenge_join_result (success = false) 를 error_code 로 나눈다
빈 결과율	explore_empty_result ÷ explore_list_view
카테고리 편중	category_grid_click 을 category 로 나눈다
방 재방문	room_view 의 주간 고유 사용자
단일 기기 정책 부작용	session_expired 를 trigger 로 나눈다
화면별 체감 속도	perf_tti 의 total_millis 를 screen_name 으로 나눈다

Amplitude 에 RuleUp 제품 지표 대시보드를 만들어 뒀다. 위 항목 중 8개가 차트로 들어가 있다. 다만 개인 스페이스에 있어 지금은 만든 사람만 볼 수 있다.

7 알려진 함정

Amplitude 에 사용자 식별자가 없다

UserIdentitySync 가 Firebase Analytics 와 Crashlytics 에만 setUserId 를 부른다. 그래서 Amplitude 의 "사용자"는 앱 설치 단위이고, 재설치·기기 교체는 새 사람으로 잡힌다. 온보딩 퍼널은 어차피 설치 단위라 영향이 작지만,

리텐션·티어처럼 계정을 따라가야 하는 지표는 지금 Amplitude 로 낼 수 없다.

Firebase 는 조용히 자르고 Amplitude 는 안 자른다

Firebase 는 이름 40자 · 키 40자 · 값 100자 · 이벤트당 파라미터 25개에서 자른다. 잘린 값은 분석에서 다른 값과 뭉치므로 매퍼가 절단 횟수를 따로 센다. 두 도구의 같은 이벤트 값이 다르면 Firebase 쪽이 잘린 것이다. Firebase 는 Boolean 을 0/1 로 바꾸고 Amplitude 는 true/false 그대로 둔다.

중복 전송이 의도인 곳이 둘

explore_filter_apply 와 explore_empty_result 는 같은 조작에서 함께 나간다 — 분모가 달라 합치면 빈 결과율을 못 낸다.

challenge_join_result 는 서버도 남기지만 클라도 남긴다 — challenge_id 를 잇는 건 클라에서만 되고, 퍼널 근거가 갈라지는 것보다 중복이 낫다.

CI 빌드에는 Amplitude 키가 없다

워크플로가 local.properties 를 만들지 않아 CI 가 뽑은 APK 는 Amplitude 출구 없이 나간다. 로컬 빌드만 키를 갖는다.

8 아직 없는 것

카탈로그가 없는 feature 다섯. verification(인증) · notification(알림) · profile(마이) · report(신고) · support(문의) 에는 이벤트 팩토리가 없다. 그 도메인의 퍼널은 만들 수 없다. 특히 인증은 제품의 중심인데 "인증 제출 → 판정 결과"를 볼 방법이 지금 없다.

새 이벤트를 추가하려면

#	할 일
---	-----

- | | |
|---|---|
| 1 | 그 도메인의 <code>observability/<Feature>Events.kt</code> 에 팩토리 함수를 만든다. 이름은 <code>snake_case</code> |
| 2 | 골든 테스트를 같이 쓴다. 이름을 바꿀 때 대시보드가 조용히 비는 대신 테스트가 깨진다 |
| 3 | ViewModel 에서 <code>observability.log(Channel.BUSINESS) { ... }</code> 로 부른다 |
| 4 | OBSERVABILITY.md 표에 한 줄 더한다. 코드와 같은 PR 에 있어야 안 썬는다 |

속성을 정할 때 둘. 모르는 값에 빈 문자열을 넣지 않는다 — 키를 빼면 결측으로 잡히고, 빈 문자열은 가짜 분류를 하나 만든다. 분모와 분자를 같은 축으로 만든다 — `challenge_id` 처럼 단계를 잇는 키가 없으면 전환율을 계산할 수 없다.

이름과 속성의 진실은 코드다. 이 문서가 코드와 어긋나면 코드가 맞다. 원본은 레포의 OBSERVABILITY.md — 이벤트를 늘리거나 이름을 바꾸는 PR 이 함께 고친다.